



Rangoli: The Real Plan

What we are actually building, how we know the numbers are right, and how we host it without spending a rupee

This picks up where the first plan and its companion math document left off. Those covered the deep color science in full. This one is about three things that plan skipped: a sharper problem worth solving, a way to prove the math is trustworthy instead of just hoping it is, and a hosting setup that costs nothing to run, ever, no matter how many people use it.

Companion to *PLAN.md* and *The Math of Color, Explained Simply* · Product, accuracy, and hosting blueprint

What's inside

- 1** What changed, and why
A generator is not the product. A fix-it tool is.
.....
- 2** The real problem
A scenario you have probably lived through, and why existing tools do not solve it
.....
- 3** The product: nine features
Diagnose, repair, generate, and the seven things built around them
.....
- 4** Making it accurate
Four separate checks, so "proven" actually means something
.....
- 5** The architecture: it all runs in your browser
Why there is no server, and why that is a feature, not a shortcut
.....
- 6** The hosting plan, step by step
What is free, what has a catch, and what you should double check yourself
.....
- 7** Where Python still lives
The research pipeline is a different system with different needs
.....
- 8** Build order
A shorter, sharper schedule than the first plan's
.....
- 9** Honest risks
What could go wrong, and what to check before you rely on any of this
.....
- 10** Names, and the plan in six lines
A short close
.....

1 What changed, and why

Two things pushed a real rethink here, not a cosmetic one. Read the whole section before jumping to the features, because it explains why the rest of this document looks different from the first plan.

The first plan treated the project as an image to palette generator with accessibility bolted on as a check afterward. That is a fine student exercise, but it walks straight into a crowded market. Coolors, Khroma, and Adobe Color already generate pretty palettes from an image or a mood, and they do it well. Being one more tool that turns a photo into five hex codes does not give anyone a reason to switch.

So the question worth asking is not "how do we generate colors." It is "what do people who already have colors actually struggle with." That turned out to have a much better answer, and it is the subject of section 2.

The second push was purely practical. You cannot pay for Vercel's paid tiers or Render's paid tiers, and that is a completely reasonable constraint, not a limitation to work around quietly. Once you take it seriously, it forces a genuinely better architecture: instead of a Python server doing the heavy math and a database remembering things, the entire engine moves into the visitor's own browser. Nobody has to keep a server running, which means nobody has to pay for one, no matter how many people use the tool. Section 5 and 6 walk through exactly how that works and where it gets hosted for free.

THE OLD PITCH

- Upload an image, get five pretty colors
- Accessibility checked after the fact, as a warning
- Needs a Python server and a database to run live
- Competes directly with Coolors and Adobe Color

THE REAL PLAN

- You already have colors. We tell you if they work, and fix them if they do not
- Accessibility is a hard rule the math has to satisfy, not a lint warning
- Runs entirely in the browser, ships as static files, costs nothing to host
- A different category: a checker and repair tool, not another generator

The good news is that almost nothing from the earlier math work gets thrown away. Every formula in the companion math document (the sRGB to Lab pipeline, delta E, k means and k medoids, WCAG, APCA) is still exactly the engine underneath. What changes is what we build around it and where it runs.

2 The real problem

Here is a scenario that is easy to picture, because it happens constantly to exactly the kind of person who would use this tool.

You are building a small product. At some point, someone (maybe you, maybe a designer, maybe a client who sent over an old brand guideline) picked three or four colors. Not from a formula, from a logo, or a gut feeling, or whatever looked good in Figma at eleven at night. You built the interface around them. Weeks later, someone runs an accessibility audit, or a screen reader user reports they cannot read your button labels, or Lighthouse flags four color pairs as failing contrast.

Now you open the design file and start nudging hex values one at a time. You darken the text color to fix the contrast against the background. That breaks the contrast against a card surface that used to be fine. You lighten the card surface. Now the primary button, which used to stand out against that surface, blends in. This can eat an entire afternoon, and at the end you are not even sure you have not broken something else you have not tested yet.

That loop is the actual problem. Not "I have no colors and want some," but "I have colors, and fixing one thing keeps breaking another thing, and I have no way to know if there even is a version of these colors that satisfies everything at once." Nobody currently answers that question directly.

Why existing tools do not solve this

Tool	What it is good at	What it does not do
Coolors, Adobe Color	Generating new palettes from a mood, an image, or a color wheel rule	Does not take your existing colors and prove a fix, does not treat contrast as a rule the output must obey
WebAIM, standalone contrast checkers	Checking one pair of colors at a time against WCAG	Only one pair at a time, no idea how changing one color affects the other five roles it also touches, no repair suggestion beyond "try again"
Tailwind or shadcn color scales	A consistent set of tints and shades once you already have a base hue	The scale itself is not checked against your actual text and background pairings, and nothing stops you from picking two shades that fail together
Figma contrast plugins	Flagging a failing pair while you design	Flags one pair at a time, same as above, and gives no guarantee that a fix for one pair will not break another

What is missing everywhere is the idea from the math document's later sections: a real interface is not five swatches, it is colors assigned to jobs (background, text, a button and its label, a danger color) with rules between the jobs, and every rule has to hold at the same time. That is a small constraint puzzle, and nobody is currently solving it as a puzzle. Most tools check pairs one at a time and hope the fixes do not fight each other.

THE ONE SENTENCE VERSION

Rangoli is not a color generator with an accessibility checkbox. It is an accessibility solver that happens to also be able to generate colors, starting from the colors you already have, whether that is a hex code, a screenshot, or a photo, and it tells you exactly what is wrong and the smallest change that fixes everything at once, with the math to back it up.

Who actually wants this

Freelance designers and small teams without a dedicated design system person, since that is exactly the gap between "picked colors that look nice" and "colors that survive an audit." Indie developers using Tailwind or shadcn who chose a brand color and never checked whether their whole token set holds together. And, quietly, anyone who has to justify a color choice to a client or a legal requirement and wants a number, not an opinion.

3 The product: nine features

Everything below runs on the same engine described in the math document. Here is what it actually looks like as something a person clicks around in.

1 Diagnose

Paste in your existing hex codes, or drop in a screenshot of your app. Rangoli finds every pairing that actually matters (body text on the background, text on a card, a button label on its own fill, a danger color against the page) and shows both the WCAG ratio and the newer APCA number for each one, side by side, with the exact margin by which it passes or fails, not just a checkmark.

rangoli · diagnose			
text on bg	WCAG 3.9:1	APCA 61	FAIL
text on surface	WCAG 4.7:1	APCA 76	PASS
onPrimary on primary	WCAG 3.1:1	APCA 58	FAIL
border on bg	WCAG 1.4:1	APCA 12	FAIL
danger on bg	WCAG 5.2:1	APCA 79	PASS

2 Repair

One click solves for the smallest possible change across every failing pair at once, using the exact constraint solve described in the math document. Not "here is a whole new palette," but "here is your palette, nudged as little as mathematically possible, with everything now passing." You see a before and after strip and the exact perceptual distance each color moved, so the fix is never a mystery.

text: before	text: after (moved 6.1)	bg: unchanged	primary: unchanged	onPrimary: before	onPrimary: after (moved 3.4)
--------------	----------------------------	---------------	-----------------------	----------------------	---------------------------------

3 Generate

For when you genuinely do not have a starting point yet. Upload a photo, a logo, or any inspiration image, and Rangoli finds the real dominant colors (the histogram and k medoids approach from the math document, so a small brand color is not lost the way random sampling would lose it) and assigns them to roles automatically. Accessible from the moment it is generated, not checked afterward.

4 Dark mode twin

Give it a light palette, get back a dark one that keeps the same hues, so it still reads as your brand, but satisfies contrast in the flipped polarity. This is a genuinely different calculation from just inverting lightness, which is exactly why APCA treats light on dark and dark on light as separate formulas in the first place.

5 Color blindness check

Simulates the common forms of color vision deficiency and flags any two roles, like your primary and danger buttons, that stop being distinguishable once simulated. Roughly one in twelve men has some form of red green color blindness, and almost no palette tool checks for it by default.

6 Export, in a format you can actually use

One click to CSS custom properties, a Tailwind config block, or a W3C design tokens JSON file. Not a screenshot of a palette you have to retype, something you paste straight into a real project.

7 A command line check for continuous integration

The same engine, packaged as a small command line tool and a matching GitHub Action, so a team can fail a pull request automatically the moment someone introduces a color pair that cannot be read. This is the feature most likely to make Rangoli part of someone's actual workflow instead of a tool they open once.

```
$ rangoli check ./tokens.json
```

```
FAIL text on bg          APCA 61  (needs 75)
```

```
FAIL onPrimary on primary APCA 58  (needs 75)
```

```
PASS text on surface     APCA 76  (needs 75)
```

```
2 pairs failing. Run "rangoli fix ./tokens.json" to resolve.
```

```
exit code 1
```

8 See the geometry

An optional 3D view of your own colors as points in the perceptual space the math actually uses, the same idea as the diagrams in the companion math document, but built from your palette instead of an example. Nice for anyone curious, and genuinely useful in a classroom or a portfolio walkthrough.

9 Share without an account

The whole palette and its role assignments get packed into the link itself, so anyone can send a result to someone else without either of you signing up for anything, and without us storing a single color anyone has ever typed in. This is a small trick, not a database, and it is enough.

WHY GENERATION IS FEATURE THREE, NOT FEATURE ONE

It would have been easy to lead with "upload an image, get a palette," because it is the most visually impressive demo. But it is also the feature every competitor already has. Leading with diagnose and repair means the first thing a visitor sees is Rangoli doing something nobody else does: taking colors that already exist and proving, with numbers, exactly what is wrong and the smallest way to fix it.

4 Making it accurate

A tool that claims to "prove" a palette is accessible only means something if the numbers behind that proof are actually correct. Here are the four separate checks this project leans on, instead of just trusting a formula because it looks right.

1. Every formula checked against a published reference

The math document already flagged this as the single highest risk in the whole project: a transposed digit in a constant produces numbers that look completely plausible and are quietly wrong, all the way through to the final palette. The fix is boring and non negotiable. Delta E 00 gets checked against Sharma's published set of test pairs, the ones specifically built to exercise every branch of the formula, including the awkward blue region. APCA gets checked against the public reference calculator. Oklab gets checked against the original author's own published constants. sRGB round trips get checked by converting thousands of random colors there and back and confirming nothing drifts. All of this runs automatically every time the code changes, not just once by hand at the start.

2. Two independent implementations, forced to agree

Because the live tool needs a fast version of this math written in JavaScript, and the research side of the project needs the same math written in Python, there is a genuine opportunity here rather than just duplicated work. Both versions get run against the same large batch of random test cases, and the build fails if they ever disagree by more than a tiny tolerance. Two independently written implementations landing on the same answer is a much stronger signal that the math is right than trusting either one alone, and this comes essentially for free once both versions exist anyway.

3. Real human taste, not just a clustering score

Whether a generated palette actually looks good to a person is a different question from whether it is mathematically tidy, and low clustering error does not automatically mean a palette looks harmonious. Rather than assume the math is right, generated palettes get scored against two published datasets built

specifically from real palettes that real people rated for quality (one from a large study of designer chosen color themes, another from a study of how people extract color themes from photographs). That gives an actual number for "does this look like something a person would choose," instead of a hopeful assumption.

4. Both accessibility standards, reported honestly

APCA is the more accurate model of how people actually read text, and the math document explains why in detail. But WCAG 2.x is still what most current laws, procurement requirements, and automated audits actually check for today. A tool that only shows the newer, better number is not more honest, it is less useful to someone who has to pass a specific audit that still cites the old one. Rangoli shows both numbers side by side, every time, rather than picking a favorite and hiding the other.

TWO SMALLER CHECKS WORTH MENTIONING

Repaired colors should still look intentional. A naive fix that just clamps numbers can shift a saturated blue toward purple, which looks like a bug even when the contrast math is correct. Repairs move through the same careful gamut mapping described in the math document, so a fixed color still looks like a deliberate design choice.

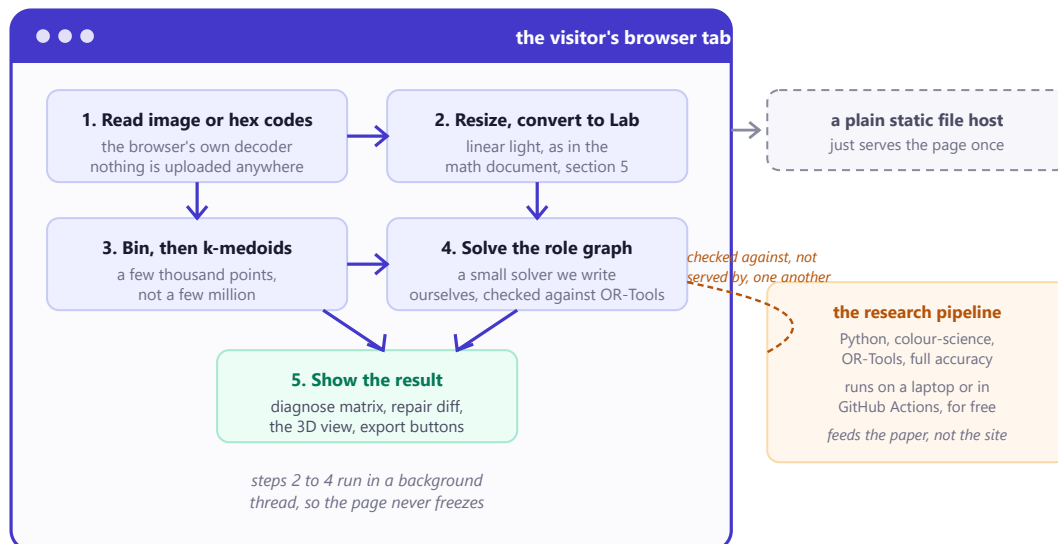
The same input always gives the same output. The clustering and the solver are both run with a fixed, seeded starting point, so uploading the same image twice gives you the same palette twice. If "proof" is going to mean anything, the tool has to be boring and repeatable, not a little different every time you run it.

5 The architecture: it all runs in your browser

This is the piece that actually solves the hosting problem, not a workaround for it. It turns out the honest answer to "we cannot pay for servers" is not "find a cheaper server," it is "do we need a server at all." For this particular tool, once you apply the binning trick from the math document, the answer is no.

Remember the performance numbers from the earlier plan: after binning an image down from a couple million pixels to a few thousand occupied buckets in perceptual space, the entire pipeline (decoding the image, converting to Lab, clustering, and solving the small constraint puzzle) runs in well under half a second. That is well within what an ordinary laptop, and even most phones, can do without breaking a sweat. There is simply not enough work here to justify a server.

So the whole engine moves client side. Here is the actual pipeline, running entirely inside the visitor's own browser tab:



Nothing about your image or your colors ever leaves your device. The static host on the right only ever hands out the same unchanging set of files, no matter how many people show up, which is exactly why it costs nothing to run at any scale a student project is likely to see.

Two details worth calling out. First, the small constraint solver that picks colors for the role graph does not need an industrial strength solver like OR-Tools, because the puzzle itself is small, a handful of roles and a few hundred candidates each. A simple solver written by hand, one that first narrows down the possibilities using the rules between roles and then searches what is left, handles this easily. The important part is that this hand written solver gets checked, on a large batch of test cases, against the same OR-Tools solver used in the research pipeline, so we know the shortcut version is not quietly wrong.

Second, this also happens to be a better product, not just a cheaper one. Nothing gets uploaded to a server, so there is no privacy question to answer about what happens to someone's screenshot or brand colors. There is no cold start delay from a sleeping free tier server waking up. And it works the same for one visitor as it does for ten thousand, because a file server genuinely does not care how much math a visitor's own browser chooses to do after downloading the page.

6 The hosting plan, step by step

Since the whole product builds down to a folder of ordinary files (HTML, JavaScript, CSS, maybe a small WebAssembly file later if it is ever needed), hosting it is genuinely simple. Here is the concrete plan, in order.

The live product

Where	Cost	Why this one
Cloudflare Pages (main recommendation)	Free	Historically a very generous free tier for static sites, no card required to start, a free subdomain immediately, automatic deploys straight from a GitHub push, and basic visit counts included so there is no need for a separate analytics tool.
GitHub Pages (simplest fallback)	Free	The code already needs to live in a public GitHub repository anyway, for the paper's reproducibility requirement. Turning that same repository into a website costs nothing extra and takes a few minutes.
Vercel Hobby tier (also fine)	Free	Not a paid plan, this is worth clarifying since it is easy to assume otherwise. It is free for personal projects, though the terms are written for individual, non commercial use, so Cloudflare Pages or GitHub Pages are the cleaner first choice.

If you later want saved palettes or a gallery

You do not need this for the core product, since sharing already works through the link itself, described back in feature nine. But if a later version wants an actual saved gallery with accounts, **Supabase's free tier** gives a real database and a login system without writing a backend server. The one thing worth knowing up front: free Supabase projects go to sleep after roughly a week without any traffic and need a manual nudge to wake back up. That is a fine tradeoff for a student project that is not yet getting daily visitors, just not something to build a paid customer's expectations around.

What is not free, and does not need to be

A custom domain, something like a made up rangoli dot design instead of a free subdomain, is usually somewhere between ten and forty dollars a year depending on the ending of the domain name. It is the one genuinely optional cost in this entire plan, and nothing about the product needs it to work. The free subdomain that Cloudflare Pages or GitHub Pages hands out is a completely real, completely working website.

CHECK THE CURRENT TERMS YOURSELF BEFORE RELYING ON ANY OF THIS

Free tiers change their limits and their rules over time, sometimes without much warning. The figures above reflect how these services have generally behaved, but five minutes spent reading a provider's current pricing page before you commit to it is five minutes well spent. Do not build a launch plan around a specific number from this document without checking it is still true.

MONTHLY COST, LIVE PRODUCT

\$0

at any traffic level a student project realistically sees

CARD REQUIRED TO START

No

for Cloudflare Pages or GitHub Pages

7 Where Python still lives

Moving the live product into the browser does not mean Python disappears from this project. It means Python has one job now, not two, and that job never needs a server.

The live tool people click around in their browser is JavaScript, and needs nothing running behind it. But the research side of this project, the parts described in the earlier plan under evaluation and ablation, still make the most sense in Python. The full sweep across five hundred or more images. The comparison tables across every combination of color space and clustering algorithm. The industrial strength OR-Tools solver, used specifically to double check the small hand written solver that ships in the browser. All of that is offline work that produces a folder of numbers, charts, and tables that go straight into the paper. None of it needs to be reachable by anyone over the internet while it runs.

It runs on the student's own laptop, or for free inside GitHub's own automated testing service, which gives a generous amount of free run time every month for exactly this kind of job, more for public repositories than private ones. Either way, nobody has to keep a Python process alive somewhere and pay for the privilege. The research pipeline and the live product are two different systems, built for two different audiences, and treating them that way is what makes the whole thing affordable.

8 Build order

Shorter than the twenty six week plan from before, because a good chunk of that plan was about running a server that no longer needs to exist. The math work carries over unchanged.

When	Focus	What ships
Weeks 1 to 2	The math core, twice	The color math from the companion document, written in TypeScript for the browser and in Python for the research side, both checked from day one against the same reference test values described in section 4.
Weeks 3 to 4	Diagnose and repair	The two headline features, working end to end as a static site, deployed to Cloudflare Pages immediately, even while it still looks rough. Getting something live early matters more than getting it polished early.
Weeks 5 to 6	Generate from an image	The binning, weighted k-medoids, and the small role graph solver, all running in a background thread so the page stays responsive.
Weeks 7 to 8	Dark mode, color blindness check, exports	The three features that make Rangoli useful inside a real project, not just a demo.
Weeks 9 to 10	The command line tool, the 3D view, share links, polish	The continuous integration story, the geometry view from the math document, and a pass over rough edges before pointing anyone at it.
Week 11 onward	The research sweep and the paper	Runs entirely separately, on a laptop and in GitHub Actions, following the evaluation plan laid out in the original document.

9 Honest risks

- **Free tier terms drift over time.** Section 6 already says this, and it is worth repeating here as a standing habit, not a one time check: read the current pricing page before leaning on any specific limit.
- **Old devices will be slower than a new laptop.** The math is light enough for almost anything modern, but a very old phone doing the full image pipeline might take a few seconds instead of a fraction of one. A visible progress indicator and a sensible cap on how large an uploaded image gets before processing handles this honestly, rather than pretending everything is instant everywhere.
- **The hand written solver is the one place a quiet bug could hide.** Because it replaces an industrial strength tool with something built from scratch for speed, it genuinely needs the differential testing described in section 4, run on a large batch of cases, not just a few hand checked examples. This is not optional polish, it is the one part of the accuracy story that has not already been proven by someone else's published work.
- **Popularity is a good problem, not a blocker.** If Rangoli ever gets popular enough that a static site is not enough on its own, for instance a gallery with real accounts and heavy write traffic, that is a genuinely nice problem to have, and nothing in this plan prevents adding a real backend later. It simply is not needed to launch, and should not be built before it is needed.

10 Names, and the plan in six lines

The names from the first plan still stand. **Rangoli** is the product, the art of geometric patterns made from colored powder, which is exactly what this tool does with a design system. **ACCORD** is the name for the underlying method when it is written up as a paper, accessibility constrained clustering for optimal role design, and it also just means harmony, which is a nice coincidence for a tool whose whole job is making colors agree with each other.

If a short line under the logo is ever needed, something honest and plain works better than anything clever: **your colors, proven readable.**

THE WHOLE DOCUMENT IN SIX LINES

1. The real problem is not generating colors, it is fixing colors people already have without breaking something else in the process.
2. Diagnose and repair come first. Generation is built on the same engine, but it is not the reason to visit.
3. Accuracy comes from checking every formula against a published reference, from two independent implementations agreeing, from real human rated data, and from reporting both accessibility standards honestly.
4. The entire engine runs inside the visitor's own browser, which is what actually solves the hosting problem, not a cheaper server.
5. Hosting is a plain static file host, genuinely free, no card required, at any traffic level this project is likely to see.
6. Python keeps exactly one job, the offline research pipeline that feeds the paper, and it never needs a server either.